In your PhD project, the data to be analysed is network, traffic, and edge-node performance data collected by your Edge Intelligence Engine. The purpose is to predict when a network path is likely to become congested and tell the Ryu SDN controller to reroute traffic before performance degrades.

You do not need sensor data to begin this stage. Your existing OVS switches already expose network statistics that you can collect.

1. What data will you analyse?

[Network scenario - Mininet - HackMD](https://images.openai.com/static-rsc-4/WdRXCHJqaLSSl3x53JMOqGiA2otM2g0XBpP_4kZTKKx9T6p5f2jTzvXzPcCLRm6ut74scr0HrLTdoCytpGZwvgvtEQCsBVYbtVs-O8qNx7e-jHMssSONnWtHi8wnPKFsZXsD8MH9f5TdEyoSzqicFyxCj5ME7GuZ8dkxlSPFxo?purpose=inline)

A. Network-link metrics — start here

Collected from OVS ports using `ovs-ofctl dump-ports`.

- RX/TX bytes and packets
- RX/TX packet drops
- RX/TX errors
- Throughput in bits per second
- Link utilisation, if link capacity is known
- Packet rate (packets per second)

[Monitor your NVIDIA Jetson IoT devices with Datadog | Datadog](https://images.openai.com/static-rsc-4/EqjFnsSc5NihyipPqa5Tr5Jw6U_zuCd20M5xSXH2kraK7Pu3civlT1ktloM7KLYm_g7B0EhP14EDOcfTZQyrkMuofsQGVt0KXnPMvnJpvnFnMF6THac0llZQC9ZdSRr0_OAyflefYC-Icp30dgE0-5J9U1rsFi4xtMUgVTi-NVQ?purpose=inline)

B. Edge-node resource metrics

Collected from Jetson6 and Jetson2.

- CPU utilisation
- Memory usage
- GPU utilisation, if available
- Network-interface utilisation
- Node readiness and failures
- Available resources for workloads

[Full article: CBFF-LSTM: a deep learning framework for early-stage DOS attack detection in IOT-enabled WSN through correlated behavioural feature fusion](https://images.openai.com/static-rsc-4/dsGrWZuCiVkDks3USAVj0sfCJAeHOD29DQZn-2_ksFSek0EPhCz0TNGpgY_l_FuOixdK5nN95JKjtSSqaNKUkr-cLStwWLgYBhCbDk47aXjtNa_T4EyyGccahPAXOofzIx92AUN2D2U64J875ZRvLLmBjPqz3c_-aPz6XzluAVY?purpose=inline)

C. End-to-end service and traffic metrics

Collected when you connect traffic sources and service endpoints.

- End-to-end latency
- Packet loss and delivery rate
- Application response time
- Traffic volume and burstiness
- Service or pod location
- Whether the destination service is healthy

2. How does this apply to your switch topology?

Your planned topology has two possible routes:

Primary route

s2

s3

s4

s7

Monitor the links s2–s3, s3–s4 and s4–s7.

Alternative route

s2

s5

s6

s4

s7

Monitor the links s2–s5, s5–s6, s6–s4 and s4–s7.

For example, suppose traffic through s3–s4 increases rapidly. The engine analyses recent throughput, packet rates and drops, along with historical observations. If the trained model predicts congestion, it can request Ryu to move traffic onto the alternative route.

Importantly, your current OVS counters are cumulative. We need to sample them repeatedly and calculate changes between samples; a single counter reading cannot tell us the current throughput.

3. What will the AI/ML model analyse?

The first model can use a dataset like this:

Timestamp	Link	Throughput	Packet rate	Drops	Congestion label
12:00:00	s3-s4	12 Mbps	1,200 pps	0	Normal
12:00:05	s3-s4	35 Mbps	3,500 pps	0	Normal
12:00:10	s3-s4	48 Mbps	5,100 pps	12	Congested

These figures are illustrative, not measurements from your cluster.

The model's input features would include throughput, packet rate, drops, errors and potentially recent trends. The target label would indicate whether the link is congested within a defined prediction window.

One important research point: the congestion labels must come from a defined, defensible rule or controlled experiments—not arbitrary guesses. For predictive routing, we should also evaluate how early the model predicts congestion and whether rerouting improves latency or packet delivery.

4. What should we collect first?

I recommend implementing the work in this order:

1. Collect OVS link metrics every 5 seconds and save timestamped records to CSV.
2. Generate controlled traffic through the virtual topology so that we have meaningful observations.
3. Add Jetson CPU, memory and network metrics.
4. Measure end-to-end latency and packet loss.
5. Generate labelled training data and train a Random Forest model.
6. Integrate predictions with Ryu so it can reroute traffic when appropriate.

For the first step, collect only the actual inter-switch ports and exclude the `LOCAL` ports, whose drop counters are not representative of inter-switch data-path loss.

This gives you a clear starting point: measure the network first, create realistic congestion scenarios second, and train the predictive model using the resulting time-series dataset.